

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 7 Exam Notes: Java I/O Streams

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Java I/O Stream Basics	2
1.1	Byte Stream vs. Character Stream	2
2	Console I/O Operations	2
2.1	Reading Console via BufferedReader	2
2.2	Reading Console via Scanner	3
3	File I/O Operations	3
3.1	FileInputStream and FileOutputStream	3
3.2	FileReader and FileWriter	3
3.3	Buffered Stream Wrapper Pattern	3

1. Java I/O Stream Basics

An Input/Output (I/O) Stream represents an abstraction of a data source (e.g., keyboard, file) or destination (e.g., console, network socket).

Stream Concept

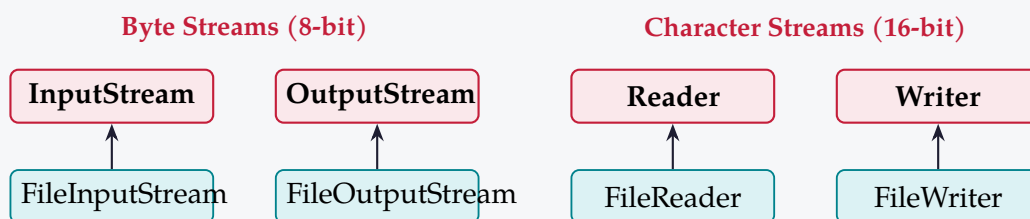
A **Stream** is a continuous sequence of data bytes. Input streams are used to read data from a source, and Output streams are used to write data to a target.

1.1 Byte Stream vs. Character Stream

Java's I/O library operates on two primary categories of streams:

- **Byte Streams:** Process data as raw 8-bit bytes. They are suitable for binary data (images, sound, zip files).
- **Character Streams:** Process data as 16-bit Unicode characters. They handle translation of text encoding models automatically.

Java Stream Hierarchy



2. Console I/O Operations

Java represents console streams using three static variables declared in the `System` class:

- **`System.in`:** Standard Input Stream (type `InputStream`), mapped to keyboard input.
- **`System.out`:** Standard Output Stream (type `PrintStream`), mapped to console output.
- **`System.err`:** Standard Error Stream (type `PrintStream`), mapped to console error output.

2.1 Reading Console via `BufferedReader`

Because `System.in` is a byte stream, we wrap it in a character stream (`InputStreamReader`) and pass it to a `BufferedReader` to enable line-by-line reading:

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
System.out.print("Enter your name: ");
String name = br.readLine();
```

2.2 Reading Console via Scanner

The `java.util.Scanner` class parses primitives and strings using regular expressions:

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter an integer: ");
int value = sc.nextInt();
```

3. File I/O Operations

3.1 FileInputStream and FileOutputStream

Used to read and write raw bytes from/to a file on disk:

```
// Copying a binary file byte-by-byte
try (FileInputStream fis = new FileInputStream("input.jpg");
    FileOutputStream fos = new FileOutputStream("output.jpg")) {
    int byteData;
    while ((byteData = fis.read()) != -1) {
        fos.write(byteData);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

3.2 FileReader and FileWriter

Used to read and write character text data from/to files on disk:

```
try (FileReader fr = new FileReader("file.txt");
    FileWriter fw = new FileWriter("copy.txt")) {
    int charData;
    while ((charData = fr.read()) != -1) {
        fw.write(charData);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

3.3 Buffered Stream Wrapper Pattern

Disk I/O requests are expensive. To improve performance, we wrap base stream classes inside buffered wrapper classes:

- **BufferedReader:** Wraps around a `Reader` (e.g., `FileReader`) to buffer characters, providing a highly efficient `readLine()` method.
- **BufferedWriter:** Wraps around a `Writer` to buffer written characters, writing to disk only when the buffer is full or when `flush()` is invoked.

Quick Revision — Module 7

Java I/O Streams | OE-EC704A

Java I/O Stream Syntax

Construct	Syntax and Example Usage
Console <code>BufferedReader</code>	<pre>BufferedReader br = new BufferedReader(new InputStreamReader(System.in));</pre>
Console Scanner	<pre>Scanner sc = new Scanner(System.in); int val = sc.nextInt();</pre>
File Reader (Text)	<pre>FileReader fr = new FileReader("input.txt"); int c = fr.read();</pre>
File Writer (Text)	<pre>FileWriter fw = new FileWriter("output.txt"); fw.write("Hello");</pre>
File Byte Input	<pre>FileInputStream fis = new FileInputStream("in.dat"); fis.read();</pre>
Buffered Writer Wrapper	<pre>BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt"));</pre>

One-Line Definitions

- **Stream:** A logical abstraction of a continuous sequence of data bytes.
- **Byte Stream:** Stream processing raw 8-bit binary data units.
- **Character Stream:** Stream processing 16-bit Unicode character data units.
- **InputStreamReader:** A bridge class converting byte streams into character streams.
- **BufferedReader:** A character stream wrapper that caches characters in memory to reduce disk calls.
- **Scanner:** A text scanner utility class parsing primitives using token delimiters.
- **System.in:** Static instance of `InputStream` mapped to the client keyboard.
- **System.out:** Static instance of `PrintStream` mapped to the standard console output.

Frequently Asked / Viva Questions

Q1. What is the difference between Byte Streams and Character Streams?

Byte streams read/write data as 8-bit bytes (base classes: `InputStream`, `OutputStream`), ideal for binary files. Character streams read/write data as 16-bit Unicode characters (base classes: `Reader`, `Writer`), ideal for text files.

Q2. Why do we wrap a `FileReader` inside a `BufferedReader`?

Reading directly from disk is slow. Wrapping in a `BufferedReader` caches character chunks in memory, reducing physical disk reads, and exposes the convenient

Frequently Asked / Viva Questions (contd.)

`readLine()` method.

Q3. What is the purpose of the `InputStreamReader` class?

It acts as a bridge, converting a byte stream (like `System.in`) into a character stream, translating bytes into characters using the default system charset.

Q4. What is the difference between `PrintStream` and `PrintWriter`?

`PrintStream` is a byte stream class handling 8-bit output (like `System.out`).
`PrintWriter` is a character stream class handling 16-bit Unicode character output.

Q5. What does the method `flush()` do in output streams?

It forces the output stream to write any cached buffered bytes/characters immediately to the destination device (like the disk file), emptying the local memory buffer.

Q6. What is the default buffer size of a `BufferedReader`?

The default buffer size is 8192 characters (8 KB), which is large enough to handle most typical text file read sizes efficiently.

Q7. How does the `Scanner` class handle delimiters?

By default, the `Scanner` class uses whitespace (spaces, tabs, newlines) as the token delimiter. This can be customized using the `useDelimiter()` method.

Q8. What is the difference between `read()` method of `InputStream` and `Reader`?

`InputStream.read()` reads a single byte and returns it as an integer (0 to 255).
`Reader.read()` reads a single Unicode character and returns it as an integer (0 to 65535). Both return `-1` when reaching EOF.

Q9. What checked exception is commonly thrown during file operations?

`java.io.IOException` and its subclass `java.io.FileNotFoundException`. They must be explicitly caught or declared using a `throws` clause.

Q10. What are the standard console stream fields inside the `System` class?

`System.in` (standard input - `InputStream`), `System.out` (standard output - `PrintStream`), and `System.err` (standard error - `PrintStream`).

Comparison: Reader/Writer vs. InputStream/OutputStream

Metric	Reader / Writer Hierarchy	InputStream / OutputStream Hierarchy
Data Unit	16-bit Unicode character unit.	8-bit raw byte unit.
Primary Target	Text documents and strings.	Images, sound, ZIP, raw binary data.
Root Packages	Lives inside <code>java.io</code>	Lives inside <code>java.io</code>
Default EOF value	Returns <code>-1</code> at stream end.	Returns <code>-1</code> at stream end.